

RG660MK-Yolo

部署设计文档(纯 CPU 方案)

任务:人脸识别 + 坐姿检测 • 目标平台:MediaTek T930 / 四核 Cortex-A55 aarch64
不依赖 NPU,基于 A55 CPU + NCNN 的边缘推理落地路径

编制:QRIBuddy(移远研究院 AI 同事)
日期:2026-08-31 • 版本 V2.0(纯 CPU 路线;性能参数需真机实测复核)

目录

TOC \h \o "1-2"

一、方案摘要

本文档针对「无 NPU」前提,给出把本地 YOLO 人脸 + 坐姿检测 demo 移植到 RG660MK 的纯 CPU 部署方案。全程只依赖模组自带的四核 Arm Cortex-A55,不使用任何 NPU/GPU 加速。

结论:可行。 demo 用的是 nano 级小模型(yolov8n 8.7 GFLOPs、yolov8n-pose 9.2 GFLOPs),且已导出为 NCNN 格式——NCNN 是对 ARM CPU(A55 的 NEON 指令)深度优化的纯 C++ 推理框架,不依赖 PyTorch,正是无 NPU 边缘设备的理想落地方式。坐姿判定是纯几何算法,与推理框架无关,可原样移植。

定位要说清: 纯 A55 CPU 上 nano 模型单帧约数百毫秒,做不到 30fps 实时,但坐姿监测本就是低速场景,通过降分辨率 + 抽帧 + 大核绑定,可稳定运行在 1~5fps,完全够用。

准确性提示:本方案的单帧耗时、帧率、内存占用均为基于 Cortex-A55 规格的工程估算,未在真机 RG660MK 上实测。上线选型前请以真机 benchmark 为准,并由硬件与算法工程师复核。

1.1 一页速览

维度	结论
源环境	x86 Intel i5-1135G7,Ultralytics 8.3 + PyTorch 2.12(CPU),内置摄像头 video0
目标环境	RG660MK:MediaTek T930(4nm),四核 Cortex-A55,OpenWrt aarch64,无 NPU
模型	yolov8n(检测)+ yolov8n-pose(姿态),均 nano 级,已导出 NCNN(约 25MB 合计)
推理后端	NCNN 纯 CPU(NEON 多线程),关闭 Vulkan/GPU
能不能直接搬	不能。x86 二进制/PyTorch 不能跑在 A55,需交叉编译 NCNN + C++ 推理程序
可行性	可行。模型已就绪,算法可照搬,1~5fps 满足坐姿监测
核心约束	A55 算力有限,需靠低分辨率 + 抽帧控制 CPU 占用,避免影响模组主业务

二、本地现有资料盘点

工作区里与本任务直接相关的一手资料如下:

2.1 YOLO demo(移植源)

文件	内容与作用
test_face_posture.py	联合测试脚本:person 检测框 + pose 17 关键点,推导人脸框,并用肩倾角/头前倾/躯干侧倾判定坐姿
export.log	模型导出日志:已把 yolov8n.pt 与 yolov8n-pose.pt 从 PyTorch →

	TorchScript → NCNN,产物已就绪
yolov8n_ncnn_model/	检测模型 NCNN 产物:model.ncnn.param + model.ncnn.bin(约 12MB)
yolov8n-pose_ncnn_model/	姿态模型 NCNN 产物:model.ncnn.param + model.ncnn.bin(约 12.7MB)
输出/*.jpg	本机摄像头跑出的标注结果图(人脸框、关键点、坐姿标签),可作视觉基线对照

关键点: demo 已经完成「PyTorch → NCNN」导出,这一步在 PC 上做好、产物直接拷到设备即可,省掉了移植中最麻烦的一环。无 NPU 场景下,NCNN 的 CPU 后端正好是标准选择。

2.2 可复用的既往经验

阶段 A 报告已实测确认 RG660MK-EU 是 OpenWrt 5.15 aarch64,并已用 aarch64-linux-gnu-gcc 成功交叉编译过 aarch64 静态可执行程序(UDP sink)。这套交叉编译 + 静态链接经验可直接用于本方案的 NCNN 推理程序构建。

三、目标平台 RG660MK 分析

3.1 硬件规格与算力定位

项目	规格
平台	MediaTek T930,4nm 制程
CPU	四核 Arm Cortex-A55(64 位 aarch64,支持 NEON SIMD)
AI 加速	本方案按「无 NPU」处理,全部计算跑在 A55 CPU 上
OS	OpenWrt(aarch64),与已验证的 RG660MK-EU 一致
定位	5G FWA 模组;本任务把它当作低功耗 aarch64 边缘计算单元使用

需确认项(不影响方案成立,但影响调优):模组可用 DDR/Flash 容量、摄像头接入方式(USB UVC / MIPI / RTSP 网络流)、CPU 主频与温控策略。建议向产品线索取 RG660MK datasheet 核实。

3.2 源环境 vs 目标环境差异

维度	源(x86 demo)	目标(RG660MK)
CPU 架构	x86-64(Intel i5)	aarch64(Cortex-A55)
推理框架	Ultralytics + PyTorch 2.12	NCNN(C++,纯 CPU)
算力量级	桌面级 4C8T @2.4GHz	嵌入式 4×A55,主频/功耗受限
内存	笔记本级(GB 富余)	模组级,需按型号确认,通常紧张
图像输入	USB 摄像头 video0	需确认接口(USB/MIPI/网络流)

OS/依赖	Ubuntu + pip 生态	OpenWrt 精简 rootfs,无 pip 重生生态
-------	-----------------	------------------------------

四、纯 CPU 部署设计

整体链路:摄像头取帧 → 抽帧 → NCNN person 检测 → 命中后 NCNN pose 出 17 关键点 → 几何算法判定坐姿/估计人脸框 → 输出结果。全部在 A55 CPU 上完成。

4.1 构建与部署步骤

1. **交叉编译 NCNN:** 用 aarch64 工具链(阶段 A 已用的 aarch64-linux-gnu-gcc)编译 NCNN,开启 NEON、关闭 Vulkan(无 GPU),尽量静态链接以适配 OpenWrt 精简环境。
2. **拷入模型:** 把 yolov8n_ncnn_model 与 yolov8n-pose_ncnn_model 两组 .param/.bin 直接拷到设备,无需再转换。
3. **编写 C++ 推理程序:** 用 NCNN 加载两个模型,实现检测 → 姿态 → 坐姿判定全链路;把 demo 的 Python 逻辑逐段翻成 C++。
4. **接入图像源:** 按设备实际摄像头接口取帧(USB UVC 用 V4L2,网络摄像头用 RTSP 拉流解码)。
5. **设置抽帧与线程:** 抽帧到 1~5fps;NCNN opt.num_threads 绑到 A55 大核;检测命中 person 后才跑 pose,避免空跑。
6. **真机 benchmark:** 实测单帧耗时/CPU 占用/内存,替换本文档的估算值,据此定最终分辨率与帧率。

4.2 坐姿算法(纯几何,原样移植)

这部分不依赖任何推理框架,把 demo 的判定规则搬到 C++ 即可,阈值保持一致:

- 肩部倾斜:双肩连线与水平夹角 $> 10^\circ$ → 高低肩
- 头前倾/含胸:鼻子相对双肩中点的垂距比 > -0.15 → 低头/含胸
- 躯干侧倾:肩中点 → 髋中点连线与竖直夹角 $> 12^\circ$ → 身体歪斜
- 人脸框:用鼻/眼/耳关键点包围盒 $\times 1.6$ 外扩估计,无需单独人脸模型
- 关键点置信度 > 0.4 才参与计算,与 demo 一致

4.3 CPU 提速手段(无 NPU 的关键)

手段	说明
降输入分辨率	640 → 416 或 320,单帧耗时可降一半以上,坐姿判定对分辨率不敏感
抽帧处理	不逐帧跑,每隔 N 帧处理一次,目标 1~5fps
两模型分时	先 person 检测,命中才跑 pose;无人时只跑轻量检测
多线程绑大核	NCNN num_threads 设为 A55 核数,避开与模组主业务抢核
INT8 量化(可选)	NCNN 支持 CPU 端 INT8,用 demo 样张做校准可再提速,需回归关键点精度

五、性能预估、工作量与风险

5.1 性能预估(工程估算,需实测)

配置(A55 纯 CPU / NCNN)	预估单帧耗时	可用帧率
640 输入,单模型	300~800 ms	约 1~3 fps
416 输入,单模型	180~450 ms	约 2~5 fps
320 输入,单模型	100~300 ms	约 3~8 fps
320 + INT8 量化	70~200 ms	约 5~12 fps

说明: 上表为单模型耗时;检测 + 姿态串行时按两者之和估。坐姿监测建议取 416 或 320 输入 + 抽帧,兼顾精度与 CPU 占用。

5.2 工作量拆解

工作项	预估工作量
交叉编译 NCNN(aarch64,NEON,静态)	0.5~1 天(工具链已具备)
C++ 推理程序 + 坐姿算法移植	2~3 天(算法照搬 demo)
图像源接入与联调	1~2 天(取决于摄像头接口)
设备端性能调优(抽帧/分辨率/线程)	1~2 天
可选:INT8 量化 + 精度回归	1~2 天

合计约 5~8 个工作日 可出真机可用版本(不含可选量化)。

5.3 风险与应对

风险	影响	应对
A55 CPU 算力有限	无法高帧率实时	降分辨率 + 抽帧 + 大核绑定;明确低速监测定位
CPU 占用挤压主业务	影响模组 5G 功能	限制处理帧率、绑核隔离、监控 CPU 温度
模组 DDR/Flash 紧张	模型/程序放不下	nano+NCNN 约 25MB,确认可用空间;必要时只保留 pose 模型
OpenWrt 缺依赖库	程序跑不起来	静态编译(参考阶段 A aarch64 静态链接经验)
INT8 量化掉精度	坐姿误判	用样张校准,量化后回归关键点偏差,阈值不达标则回退 FP

六、下一步建议

7. 确认摄像头接口与模组 DDR/Flash 容量,这决定取帧方式和是否需裁模型。
 8. 先交叉编译 NCNN 并在真机跑通一个模型,实测单帧耗时,校准本文档估算值。
 9. 移植坐姿算法、串起完整链路,用 demo 样张比对结果一致性。
 10. 按真机数据定分辨率/帧率,必要时上 INT8 量化;确定常驻服务还是按需触发。
- 需要我直接把交叉编译脚本和 C++ 推理骨架写出来,说一声即可。

附:本方案依据的本地资料 — `yolo_demo/test_face_posture.py`、`yolo_demo/export.log`、RG660MK 阶段 A 报告;平台规格来自移远官网 RG660MK 系列产品页。性能数字为工程估算,上线前请以真机实测为准。